



Assessment Cover Sheet

Assessment Title	Project_Group		
Assessment Type	Uncontrolled	Group	Not must-pass
Due Date	15-12-2025	Course Code	IT7009
Course Title	Artificial Intelligence		
Internal Moderator's Name	Mahmoud Alhamad		
External Examiner's Name			

Instructions:

1. This cover sheet must be completed (section in blue below) and attached to your assessment before submission in soft copy.
2. The deadline for this assessment is 15-Dec-2025 (11:55 PM).
3. This assessment carries 35% marks distributed to a total of 3 questions assessing CILO 1, CILO2, and CILO 3.
4. Late submission, marks will be capped at 60%. Two calendar days after the deadline, students will get 0%.
5. References consulted (if any) must be properly acknowledged and cited.
6. The assessment has a total of 5 pages.

Learner ID	202301660 - 202302130 - 202301089 - 202305220 - 202200435	Date Submitted	31 - 12-2025
Learner Name	Norain Almajed - Raghad Aleskafi - Fatema Alaiwi - Zahra Hubail - Jumana Ali		
Programme Code			
Programme Title			
Lecturer's Name	Sharifa S.Ali		

By submitting this assessment for marking, I affirm that this assessment is my own work.

Do not write beyond this line. For assessor use only.

Learner Signature			
Assessor's Name			
Marking Date		Marks Obtained	

Comments:

IT7009 PROJECT

- Norain Almajed-202301660
- Raghad Aleskafi- 202302130
- Fatema Alaiwi- 202301089
- Zahraa Hubail-202305220
- Jumana Ali-202200435

Table Of Contents

Table Of Contents	3
Introduction	7
- Overview of the project and its purpose	7
- Summary of dataset and tasks to be carried out in the project	7
Task Sections	8
Dataset Preprocessing	8
Unsupervised Learning	15
Detailed Methodology (Design Decisions)	15
- Feature Development	15
- Data preparation	16
Performance Results & Critical Comparison	17
- Model Design – Elbow Method	17
- Critical Comparison	19
Interpretation of Model Behavior	20
- Validation	20
Supervised Learning (Regression)	23
Detailed Methodology (Design Decisions)	23
- Data Preparation (Filtering Tickers)	23
- Feature Development	24
Creating the Model	25
- Data splitting and scaling	25
- Model Design	26
Interpretation of Model Behavior	27
- Model Evaluation	27
- Visualization	29
Discussion on the feasibility and limitations of deploying such a system in practice.	31
- Summary	32
Unsupervised Learning Summary	32
Supervised Learning Summary	32
- Challenges	33

-	Recommendations	34
	References	35

List of Figures

Figure 1. Sampling Dataset.....	8
Figure 2. Checking duplicates values	8
Figure 3. Checking null values	9
Figure 4. Clean Column by removing infinity Values.....	10
Figure 5. Standardize Text Formatting	10
Figure 6. Data Type Conversion.....	11
Figure 7. Fixing incorrect Values	12
Figure 8. Fixing Incorrect Values	12
Figure 9. Outlier Detection (1).....	12
Figure 10. Outlier Detection (2).....	13
Figure 11. Outlier Detection (3).....	13
Figure 12. Outlier Detection (4).....	13
Figure 13. Outlier Detection (5).....	13
Figure 14. Outlier Detection (6).....	14
Figure 15. Outlier Detection (7).....	14
Figure 16. Encoding Categorical Data.....	14
Figure 17: Feature Engineering For Unsupervised Learning.....	15
Figure 18: Cluster Selection (Elbow Method)	17
Figure 19: K-Means Clustering for $k = 3$	20
Figure 20: Outlier refinement and feature rescaling prior to final clustering.	21
Figure 21: Final K-Means clustering results for stock risk profiles.....	22
Figure 22: Sorting ticker values by date	23
Figure 23:Filtering Data using quantity and quality checks	24
Figure 24: Log-Transforming closing prices	24
Figure 25: Engineering new features based on the past trading week and month prices.....	25
Figure 26: setting the training and testing data	25
Figure 27: splitting the training and testing data.....	25
Figure 28:Scaling input features	26
Figure 29: Creating the linear regression model	26
Figure 30:Predicting stock prices in log-space	26
Figure 31:back-transforming prediction values	27
Figure 32:scatter plot with (log-log) scale to show Actual vs. Predicted Prices.....	29
Figure 33:Confusion Matrix for Experimental Supervised Learning (classification) Model	30

List of Tables

Table 1: Data Preparation and Feature Scaling	16
Table 2: K-Means cluster assignments for different values of k.	18
Table 3: Silhouette and Davies–Bouldin scores for different cluster configurations.....	19
Table 4: Mean, Median and count for stocks across each cluster.....	21
Table 5: Final stock count per cluster after rescaling and outliers removal	22
Table 6: Average coefficients and intercept for model.....	27
Table 7: Model Evaluation report using 4 evaluation metrics	28

Introduction

- Overview of the project and its purpose

The stock market produces large volumes of historical trading data that capture daily price movements, trading activity, and market behavior across thousands of stocks. Analyzing this data is challenging due to its size, noise, missing values, and abnormal events such as stock splits and extreme price changes. These challenges make traditional analytical methods less effective and motivate the use of machine learning techniques for extracting meaningful insights.

This project focuses on analyzing global stock market data using machine learning to understand stock behavior and predict future price movements. The dataset consists of daily historical records containing numerical features such as opening price, highest price, lowest price, closing price, trading volume, dividends, and stock split information. Due to the scale of the dataset, a subset was extracted and thoroughly preprocessed to ensure data quality and consistency.

- Summary of dataset and tasks to be carried out in the project

The proposed solution implements a complete machine learning pipeline that integrates data preprocessing, unsupervised learning, and supervised learning. Data cleaning steps include removing duplicate records, handling missing and invalid values, standardizing formats, encoding categorical variables, and scaling numerical features. For exploratory analysis, unsupervised learning is applied using K-Means clustering on volatility and maximum drawdown features to group stocks with similar risk and return characteristics. Clustering quality is evaluated using established metrics and visualized for interpretation.

In addition, a supervised regression-based model is developed to predict future stock prices using historical information and engineered technical indicators such as lagged prices, moving averages, and daily returns. The model is evaluated using standard performance metrics to assess prediction accuracy. Overall, the project demonstrates how machine learning techniques can be effectively applied to analyze complex stock market data and support data-driven financial analysis.

Task Sections

Dataset Preprocessing

```
'''import pandas as pd

# Define input and output file names
input_csv_file = '/content/drive/MyDrive/all_stock_data.csv'
output_csv_file = 'first_10mil_records.csv'

# Read the first 5000 rows (including header if present)
df = pd.read_csv(input_csv_file, nrows=10000000)

# Write the DataFrame to a new CSV file
# 'index=False' prevents pandas from writing the DataFrame index as a column
df.to_csv(output_csv_file, index=False)

print(f"Successfully extracted the first 10M records from '{input_csv_file}' to '{output_csv_file}'.")'''
```

Figure 1. Sampling Dataset

As shown in Figure 1, due to the large size of the original dataset, which contains approximately 34 million records, it was not practical to process the entire data on standard personal laptops because of memory and performance limitations. To address this issue, we applied a sampling strategy by extracting the first 10 million records and saving them into a new dataset. This approach significantly reduced computational overhead while still preserving a very large number of observations. The selected sample size remains sufficiently extensive to capture underlying patterns, trends, and relationships within the data, ensuring that the analytical and modeling results remain reliable and meaningful. By sampling the dataset in this manner, we achieved a balance between computational feasibility and data representativeness, allowing the project to be executed efficiently without compromising the quality of the analysis.

```
Duplicated Values

#Checking the number of rows before dropping the duplicated rows
import pandas as pd
print(df.shape)

#Dropping the duplicated rows in the database.
df.drop_duplicates(inplace = True)

#Checking the number of rows after dropping the duplicated rows
print(df.shape)
```

Figure 2. Checking duplicates values

As shown in Figure 2, The first step after reviewing the dataset is to see a total of 34,646,258 rows and 9 columns. Once the duplicate row check was completed and repeated entries were removed, the dataset size remained the same. This verifies that the dataset did not include any duplicate copies and already had clean duplicates.

```
2. Null Values

#Checking the null values
print("Data before dropping null values:")
print(df.isnull().sum())

df.dropna(inplace = True)
df.dropna(axis = 1, inplace = True)

print("\n\nData after dropping null values:")
print(df.isnull().sum())

Data before dropping null values:
Date      0
Ticker     0
Open     109
High     109
Low      109
Close     106
Volume     0
Dividends  0
Stock Splits  0
dtype: int64

Data after dropping null values:
Date      0
Ticker     0
Open      0
High      0
Low       0
Close     0
Volume     0
Dividends  0
Stock Splits  0
dtype: int64
```

Figure 3. Checking null values

As shown in Figure 3, The dataset was first screened by null value checking, and in the preliminary output, we can observe some columns that are missing data. All rows with null values were removed using a row-based deletion technique to confirm that the dataset was clean and suitable for analysis. Columns with missing values after this step were also completely removed. Following these cleaning operations, a second check verified that in our dataset we no longer had null values, so this data was preprocessed and complete.

2.1. Clean 'Stock Splits' Column by Removing Infinity Values

```
import numpy as np

# Select all numeric columns
numeric_cols = df.select_dtypes(include=['number']).columns

# Check for infinity values before cleaning
print("Before cleaning:")
print(np.isinf(df[numeric_cols]).any())

# Replace infinity in 'Stock Splits' with NaN
df["Stock Splits"] = df["Stock Splits"].replace([np.inf, -np.inf], np.nan)

# Drop rows where 'Stock Splits' is NaN
df = df.dropna(subset=["Stock Splits"])

# Check for infinity values after cleaning
print("\n\nAfter cleaning:")
print(np.isinf(df[numeric_cols]).any())
```

Figure 4. Clean Column by removing infinity Values

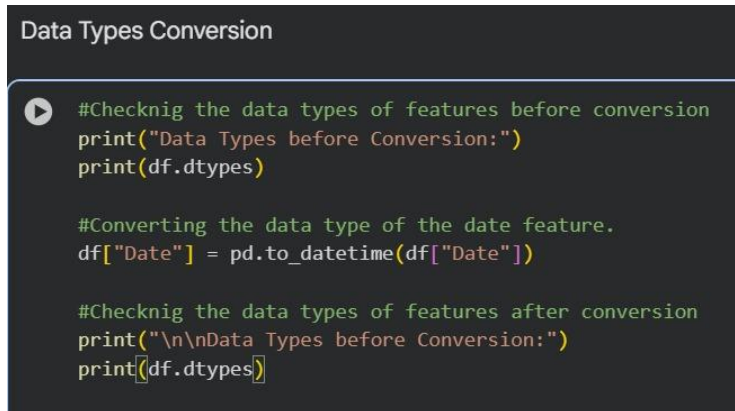
As shown in Figure 4, the numeric features were examined for the presence of infinite values. The data shows we have clean columns except for Stock Splits with an infinite value. We replaced it with a missing value and removed this value from the dataset. After sanitization, the second check verified that there were no infinite values in any numeric feature, thus the dataset is valid and ready to be subjected to further processing.

Standardize text formatting

```
#Standardize text formatting
#Converting the Ticker feature data to upper case.
df["Ticker"] = df["Ticker"].str.upper()
#remove extra spaces from Ticker feature data using strip.
df["Ticker"] = df["Ticker"].str.strip()
```

Figure 5. Standardize Text Formatting

As shown in Figure 5, the Ticker column in the dataset was standardized for consistency. All ticker symbols were given the same form (uppercase). Other spaces that were added at the start or end of the ticker values were deleted. This avoids problems in which slightly different text formats (lowercase symbols or unnecessary spaces) can become classified as different categories. All the Ticker values of the dataset were cleaned following our formatting methodologies.



```
#Checknig the data types of features before conversion
print("Data Types before Conversion:")
print(df.dtypes)

#Converting the data type of the date feature.
df["Date"] = pd.to_datetime(df["Date"])

#Checknig the data types of features after conversion
print("\n\nData Types before Conversion:")
print(df.dtypes)
```

Figure 6. Data Type Conversion

As shown in Figure 6, The data of all features were first investigated and checked data types of features. On the former, Date column was a type of text, not a new date. To apply in time-related operations the Date column was transformed to datetime form properly. The second post-transformation check confirmed we converted to datetime64 data type in the Date column and all the other feature types stayed constant. This ensures that the dataset can handle time-based analysis and future feature extraction properly. The results showed a lot of rows that had nothing or negative prices, invalid in real stock market data. In detail, records with purely positive values were retained, and rows with zero or negative prices were discarded.

```

5. Fixing Incorrect Values

#Checking incorrect values in Volume, Dividends and Stock Splits features
print("Checking zero values in Open, High, Low and Close features before changing:\n")
print(
    df[
        (df["Open"] <= 0) |
        (df["High"] <= 0) |
        (df["Low"] <= 0) |
        (df["Close"] <= 0)
    ]
)

#Saving only values greater than 0
df = df[
    (df["Open"] > 0) &
    (df["High"] > 0) &
    (df["Low"] > 0) &
    (df["Close"] > 0)
]

#Checking after the changes
print("\n\nChecking zero values in Open, High, Low and Close features after changing:\n")
print(
    df[
        (df["Open"] <= 0) |
        (df["High"] <= 0) |
        (df["Low"] <= 0) |
        (df["Close"] <= 0)
    ]
)

```

Figure 7. Fixing incorrect Values

As shown in Figure 7, The dataset was checked for incorrect values in the Price features (Open, High, Low and Close). As shown in Figure 7, The dataset was checked for incorrect values in the Price features (Open, High, Low and Close). Many rows of negative or zero prices appeared in the results that simply don't belong to real stock market data. For accuracy's sake only records with strictly positive values were kept while zero or negative prices were excluded. Following the cleaning process, a second step verified that no invalid values were left in the price features, which means the dataset remains quite consistent and reliable.

```

Checking Incorrect Values

#Checking incorrect values in Volume, Dividends and Stock Splits features
print(
    df[
        (df["Volume"] < 0) |
        (df["Dividends"] < 0) |
        (df["Stock Splits"] < 0)
    ]
)

```

Figure 8. Fixing Incorrect Values

As shown in Figure 7, the Volume, Dividends and Stock Splits features were checked for negative values. As shown in Figure 8, the Volume, Dividends and Stock Splits features were checked for negative values. These columns should never have a negative value because trading volume cannot be negative, nor dividends or

```

7.1. Outliers in Open Feature

#Checking outliers in Open Feature

Q1 = df["Open"].quantile(0.25)
Q3 = df["Open"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Open"] < lower_bound) | (df["Open"] > upper_bound)]
print(outliers)

```

Figure 9. Outlier Detection (1)

```

7.2. Outliers in High Feature

#Checking outliers in High Feature

Q1 = df["High"].quantile(0.25)
Q3 = df["High"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["High"] < lower_bound) | (df["High"] > upper_bound)]
print(outliers)

```

Figure 10. Outlier Detection (2)

```

7.3. Outliers in Low Feature

#Checking outliers in Low Feature

Q1 = df["Low"].quantile(0.25)
Q3 = df["Low"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Low"] < lower_bound) | (df["Low"] > upper_bound)]
print(outliers)

```

Figure 11. Outlier Detection (3)

```

7.4. Outliers in Close Feature

#Checking outliers in Low Feature

Q1 = df["Close"].quantile(0.25)
Q3 = df["Close"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Close"] < lower_bound) | (df["Close"] > upper_bound)]
print(outliers)

```

Figure 12. Outlier Detection (4)

```

7.5 Outliers in Volume Feature

#Checking outliers in Volume Feature

Q1 = df["Volume"].quantile(0.25)
Q3 = df["Volume"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Volume"] < lower_bound) | (df["Volume"] > upper_bound)]
print(outliers)

```

Figure 13. Outlier Detection (5)

```

7.6. Outliers in Dividends Feature

#Checking outliers in Dividends Feature

Q1 = df["Dividends"].quantile(0.25)
Q3 = df["Dividends"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Dividends"] < lower_bound) | (df["Dividends"] > upper_bound)]
print(outliers)

```

Figure 14. Outlier Detection (6)

```

7.7. Outliers in Stock Splits Feature

#Checking outliers in Stock Splits Feature

Q1 = df["Stock Splits"].quantile(0.25)
Q3 = df["Stock Splits"].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = df[(df["Stock Splits"] < lower_bound) | (df["Stock Splits"] > upper_bound)]
print(outliers)

```

Figure 15. Outlier Detection (7)

As shown in Figures 9, 10, 11, 12, 13, 14 and 15, outliers were detected across all numerical features using the IQR method. The price-related features (Open, High, Low, and Close) showed a very large number of outliers, mainly because the dataset contains stocks with extremely different price ranges from very low-priced stocks to companies with prices in the hundreds or even thousands. The Volume feature also produced many outliers, which is expected in real market data since some stocks are traded far more frequently than others. For the Dividends and Stock Splits features, outliers appeared only in rows where values were unusually high compared to the rest of the dataset. Overall, the detected outliers represent real market behavior rather than errors, so they were documented but not removed.

8. Encoding Categorical Data

```

#Encoding the categorical features into numbers using Label Encoder.
from sklearn.preprocessing import LabelEncoder

#Data before encoding
print("Data before encoding")
print(df.head())

encoder = LabelEncoder()
df["Ticker"] = encoder.fit_transform(df["Ticker"])

#Data after encoding
print("\n\nData before encoding")
print(df.head())

```

Figure 16. Encoding Categorical Data

[As shown in Figure 15, the column "Ticker" with categorical stock symbols was transformed into numeric values to accommodate the model-building process.](#) As shown in Figure 16, the column "Ticker" with categorical stock symbols was transformed into numeric values to accommodate the model-building process. Using Label Encoding, each unique Ticker was assigned a numerical label. It was selected as a solution since it is an efficient approach to store all information about the category and it works well for nominal features

Unsupervised Learning

The main goal of the unsupervised learning task was to identify groups of stocks with similar risk behavior without using predefined labels. This was achieved by clustering stocks based on volatility and maximum drawdown derived from historical price data. K-Means clustering was used to uncover natural risk-based groupings in the dataset. The results help in understanding different stock risk profiles in the market.

Detailed Methodology (Design Decisions)

- Feature Development

```
import numpy as np
import pandas as pd

# get unique stock identifiers
tickers = df['Ticker'].unique()

features = []

# Process each stock separately to avoid mixing time series
for ticker in tickers:

    g = df[df['Ticker'] == ticker][['Date', 'Close']].copy()
    g['Date'] = pd.to_datetime(g['Date'])
    g = g.sort_values('Date')
    # Skip stocks with insufficient data (cannot compute returns)
    if len(g) < 2:
        continue

    # Compute daily returns based on closing prices
    g['Prev_Close'] = g['Close'].shift(1)
    g['Return'] = (g['Close'] - g['Prev_Close']) / g['Prev_Close']
    # Handle infinite and missing values caused by price anomalies
    g['Return'] = g['Return'].replace([np.inf, -np.inf], np.nan).fillna(0)

    # Volatility: standard deviation of daily returns
    # Higher values indicate higher risk and instability
    std_r = g['Return'].std()

    # Maximum drawdown: worst percentage loss from a historical peak
    roll_max = g['Close'].cummax()
    drawdown = (g['Close'] / roll_max) - 1
    max_dd = drawdown.min()

    features.append({
        'Ticker': ticker,
        'std_return': std_r,
        'max_drawdown': max_dd
    })

features_df = pd.DataFrame(features)
features_df.head(), features_df.describe()
```

Figure 17: Feature Engineering For Unsupervised Learning

Figure 17 illustrates the feature extraction process used to prepare the dataset for unsupervised learning. Each stock was processed independently to preserve the integrity of its time-series data and avoid mixing price information across different tickers. Historical closing prices were first sorted chronologically, and stocks with insufficient data were excluded, as return calculations require at least two observations. Daily returns were then computed using consecutive closing prices. Any infinite or missing return values caused by price anomalies were handled to ensure numerical stability. From these returns, two key risk-related features were derived: volatility, measured as the standard deviation of daily returns, and maximum drawdown, representing the worst observed percentage loss from a historical peak. These features were aggregated at the stock level to produce a compact dataset where each row represents a single stock. The resulting feature set formed the input for the clustering algorithm, enabling the identification of stocks with similar risk behavior.

The initial dataset was significantly larger and caused memory limitations during feature extraction, leading to long execution times and RAM crashes. The issue was resolved by reducing the dataset size and aggregating features at the stock level, enabling efficient and stable analysis.

- Data preparation

Table 1: Data Preparation and Feature Scaling

<pre> from sklearn.preprocessing import StandardScaler # Select only numerical features for clustering X = features_df[['std_return', 'max_drawdown']] # Initialize scaler scaler = StandardScaler() # Scale features X_scaled = scaler.fit_transform(X) # Convert back to DataFrame (for readability) X_scaled_df = pd.DataFrame(X_scaled, columns=['std_return_scaled', 'max_drawdown_scaled']) X_scaled_df.head() </pre>		
	std_return_scaled	max_drawdown_scaled
0	-0.018377	-0.780918
1	-0.018385	0.012090
2	-0.018376	-0.878040
3	-0.018383	-0.197383
4	-0.018381	0.270065

To ensure fair distance computation in K-Means clustering, volatility and maximum drawdown were standardized using the StandardScaler after feature engineering, resulting in normalized input features suitable for clustering.

Performance Results & Critical Comparison

- Model Design – Elbow Method

```
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
# measures how tight clusters are
inertia = []

K_range = range(2, 11)

for k in K_range:
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    kmeans.fit(X_scaled_df)
    inertia.append(kmeans.inertia_)

plt.plot(K_range, inertia, marker='o')
plt.xlabel('Number of Clusters (k)')
plt.ylabel('Inertia')
plt.title('Elbow Method for Optimal k')
plt.show()
```

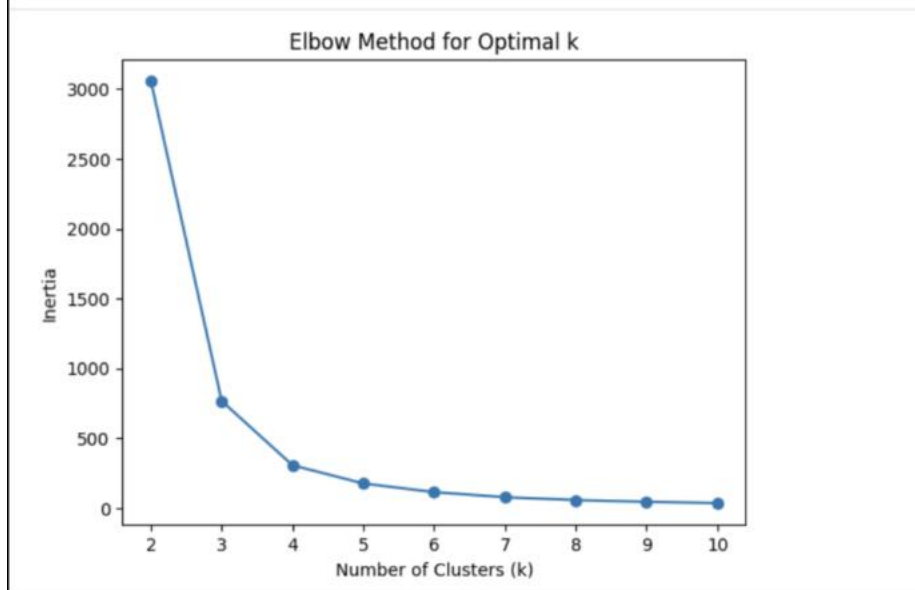


Figure 18: Cluster Selection (Elbow Method)

Figure 18 demonstrates the graph of the elbow method, which is used to evaluate different values of k, and the clear change in inertia reduction around k = 3 indicated an appropriate number of clusters.

Table 2: K-Means cluster assignments for different values of k .

<pre> from sklearn.cluster import KMeans k_values = [3, 4, 7] for k in k_values: kmeans = KMeans(n_clusters=k, random_state=42, n_init=10) clusters = kmeans.fit_predict(X_scaled_df) features_df[f'Cluster_k{k}'] = clusters features_df.head() </pre>						
	Ticker	std_return	max_drawdown	Cluster_k3	Cluster_k4	Cluster_k7
0	771	0.020505	-0.856558	0	0	1
1	1171	0.015164	-0.630440	0	1	6
2	258	0.020809	-0.884251	0	0	1
3	1398	0.016222	-0.690169	0	1	6
4	454	0.018014	-0.556881	2	1	6

After determining suitable values of k , K-Means clustering was applied using multiple cluster configurations ($k = 3, 4$, and 7). For each value of k , the algorithm assigned a cluster label to every stock based on the scaled feature set.

- Critical Comparison

Table 3: Silhouette and Davies–Bouldin scores for different cluster configurations

<pre> from sklearn.metrics import silhouette_score, davies_bouldin_score evaluation_results = [] k_values = [3, 4, 7] for k in k_values: labels = features_df[f'Cluster_k{k}'] sil_score = silhouette_score(X_scaled_df, labels) db_score = davies_bouldin_score(X_scaled_df, labels) evaluation_results.append({ 'k': k, 'Silhouette_Score': sil_score, 'Davies_Bouldin_Index': db_score }) evaluation_df = pd.DataFrame(evaluation_results) evaluation_df </pre>			
	k	Silhouette_Score	Davies_Bouldin_Index
0	3	0.631588	0.338949
1	4	0.626848	0.355817
2	7	0.584004	0.430845

Based on table 3, we evaluated the quality of the clustering results, the Silhouette Score and Davies–Bouldin Index were computed for different values of k. The Silhouette Score measures how well-separated the clusters are, while the Davies–Bouldin Index evaluates cluster compactness and separation. As shown in Table 24, k = 3 achieved a strong balance between high silhouette score and low Davies–Bouldin index, indicating well-defined and stable clusters compared to higher values of k.

Interpretation of Model Behavior

- Validation

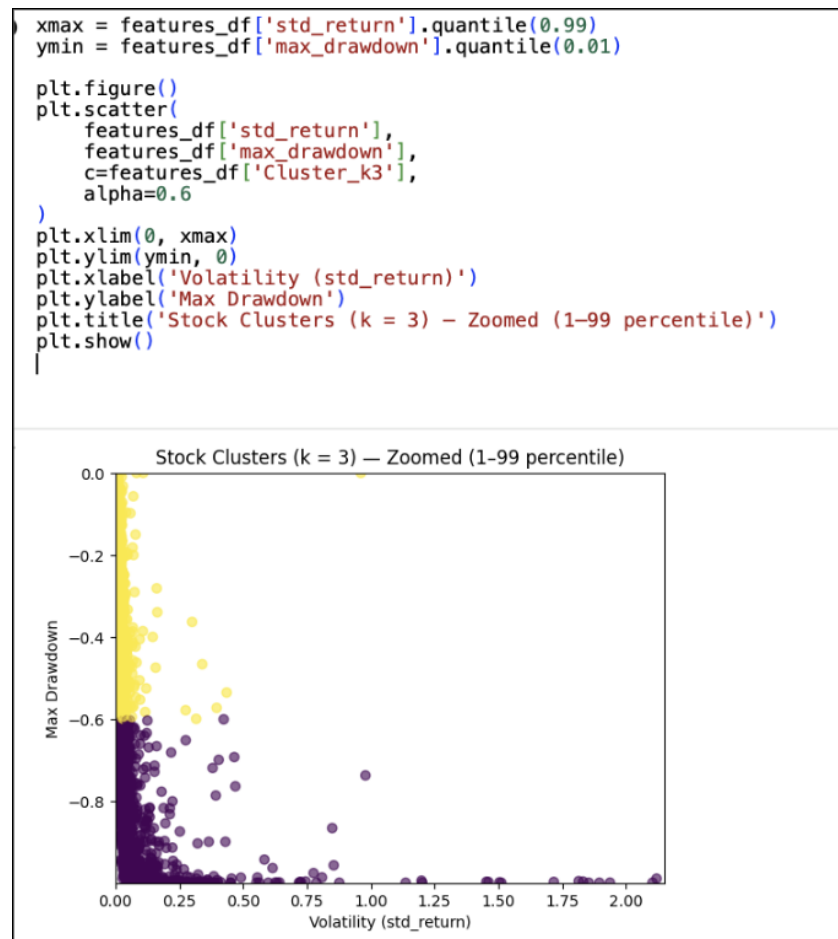


Figure 19: K-Means Clustering for $k = 3$

Figure 19 presents the clustering result for $k = 3$ using volatility and maximum drawdown as input features. To improve visual clarity, the plot was zoomed to exclude extreme values by limiting the display to the 1st and 99th percentiles. This visualization allows for clearer inspection of the main concentration of stocks and their relative separation in the risk feature space.

Although $k = 3$ was selected based on both the elbow method and quantitative evaluation metrics, the visual inspection in Figure 18 appears to show only two dominant groups. This occurs because one cluster contains a relatively small number of stocks compared to the others and is therefore less visually prominent in the scatter plot. To further investigate this behavior, the distribution of cluster assignments was examined by counting the number of stocks in each cluster, which confirmed the presence of three distinct clusters despite the visual overlap.

Table 4: Mean, Median and count for stocks across each cluster

features_df.groupby('Cluster_k3')[['std_return', 'max_drawdown']].agg(['mean', 'median', 'count'])						
Cluster_k3	std_return			max_drawdown		
	mean	median	count	mean	median	count
0	0.346344	0.045321	1731	-0.849985	-0.878049	1731
1	35842.726045	35842.726045	1	-0.798780	-0.798780	1
2	0.033757	0.018619	1327	-0.351875	-0.360659	1327

Table 4 shows the distribution of stocks across clusters for $k = 3$. While two clusters contain a large number of stocks, one cluster consists of a single stock. This occurs because the stock exhibits exceptionally high volatility compared to the rest of the dataset, leading K-Means to isolate it as a separate cluster. This result reflects the sensitivity of K-Means to extreme values in financial data.

```
# Handle extreme outliers (final refinement)
thr = features_df['std_return'].quantile(0.958)
features_clean = features_df[features_df['std_return'] <= thr].copy()
features_clean.shape

(2930, 6)

from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_clean_scaled = scaler.fit_transform(
    features_clean[['std_return', 'max_drawdown']]
)
X_clean_scaled_df = pd.DataFrame(
    X_clean_scaled,
    columns=['std_return_scaled', 'max_drawdown_scaled'],
    index=features_clean.index
)
```

Figure 20: Outlier refinement and feature rescaling prior to final clustering.

Figure 20 shows the final refinement step, where extreme volatility outliers were removed using a percentile threshold of 0.958 to reduce their influence on clustering. The remaining features were then rescaled to ensure proper normalization before applying K-Means. Due to outliers removal, feature rescaling was necessary to ensure that the final input to the clustering algorithm was properly normalized and not influenced by the previous scaling parameters.

Table 5: Final stock count per cluster after rescaling and outliers removal

```
from sklearn.cluster import KMeans

kmeans3 = KMeans(n_clusters=3, random_state=42, n_init=10)

features_clean['Cluster_k3'] = kmeans3.fit_predict(X_clean_scaled_df)

features_clean['Cluster_k3'].value_counts()
```

	count
Cluster_k3	
0	1364
1	1341
2	225

As shown in table 5, the number of stocks assigned to each cluster after removing extreme volatility outliers and reapplying K-Means with $k = 3$. Compared to earlier results, the clusters are more balanced, indicating improved stability and reduced influence of extreme values on the clustering process.

Although one cluster remains smaller than the others, this is expected in financial data, where a subset of stocks exhibits distinct risk characteristics.

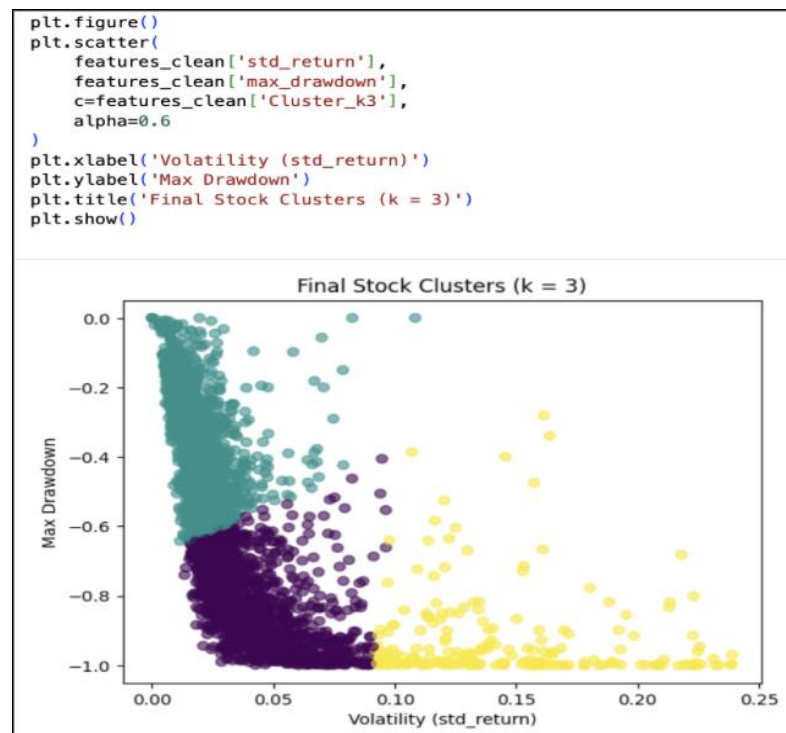


Figure 21: Final K-Means clustering results for stock risk profiles

Figure 21 visualizes the final clustering of stocks based on volatility (standard deviation of returns) and maximum drawdown after outlier removal. Each color represents a different cluster identified by K-Means. The first cluster (dark purple) corresponds to low-volatility stocks with relatively moderate drawdowns, indicating more stable performance. The second cluster (teal) represents stocks with moderate volatility and higher drawdowns, reflecting medium-risk behavior. The third cluster (yellow) captures high-volatility stocks with severe drawdowns close to -1 , indicating higher risk.

The vertical concentration of points at low volatility values and the uneven spacing between clusters are expected due to the skewed distribution of financial risk metrics, where most stocks exhibit low volatility while fewer stocks show extreme behavior. This confirms that the clustering reflects meaningful risk segmentation rather than visual distortion.

Supervised Learning (Regression)

This Supervised Learning model trains the machine to guess the next closing price based on historic closing data (the last trading day, the last 5 trading days, the last 20 trading days ~trading month~). Please note that this model is only a baseline model, and even with the high evaluation values, it is nearly impossible to guess stock prices accurately in this high accuracy in the real world due to external factors and market fluctuations.

Detailed Methodology (Design Decisions)

- Data Preparation (Filtering Tickers)

```
unique_tickers = df['Ticker'].unique()
scanned_tickers = len(unique_tickers)

for ticker in unique_tickers:
    #ensure that ticker is in chronological order
    df_ticker = df[df['Ticker'] == ticker].copy().sort_values('Date')
```

Figure 22: Sorting ticker values by date

As shown in Figure 22, To make the model accurate and ensure that it only trains on historical data, we started by sorting each Ticker value by date. To avoid mixing up future and past data.

```
#11.2.2 Quantity & Quality Filters

#Quantity Check: Remove tickers with too few records for meaningful training
if len(df_ticker) < 100: continue
eligible_quantity_tickers += 1 #Data cleaning tracking

#Quality Check: Detect and remove tickers with data corruption or unadjusted splits
#A >50% jump in one trading day is usually a non-market event (split/error).
if df_ticker['Close'].pct_change().abs().max() > 0.50: continue
eligible_quality_tickers += 1 #Data cleaning tracking
```

Figure 23: Filtering Data using quantity and quality checks

As shown in Figure 23, we had to filter through the tickers and make sure that the model trains on valid values based on the following quantity and quality checks:

Quantity check:

For each unique ticker in the ticker column, we made sure that each ticker had enough records for the model to accurately train on, 100 records in this case, and any ticker with less than 100 records was skipped.

Quality check:

To make the model closer to the real world, we filtered out any models with more than a 50% jump in closing price within one trading day, because this huge jump is usually a non-market event and probably an error or split. Any ticker with a huge jump was skipped to avoid model errors.

-Feature Development

```
# Log-transform prices to linearize exponential growth and stabilize variance
df_ticker['Log_Close'] = np.log(df_ticker['Close'])
```

Figure 24: Log-Transforming closing prices

As shown in Figure 24, we implemented Log-Transform:

Since stock values can range from one to thousands of dollars, we used log-transform on the 'close' column in our dataset, this is the standard way to study stock returns according to *Hudson and Gregoriou (2015)*, because log-transform stabilizes the data and treats absolute price changes as change percentages, so that the model treats all price movements fairly, e.g. A 1% change in a low stock could be a few dollars or even cents, but it could be hundreds or thousands in higher priced stocks.

```

# We must 'Shift' features by 1 day so the model predicts tomorrow using ONLY yesterday's data.
df_ticker['Last_Close'] = df_ticker['Log_Close'].shift(1) # Previous Price
df_ticker['SMA_5'] = df_ticker['Log_Close'].shift(1).rolling(window=5).mean() # Weekly Trend
df_ticker['SMA_20'] = df_ticker['Log_Close'].shift(1).rolling(window=20).mean() # Monthly Trend
df_ticker['Daily_Return'] = df_ticker['Log_Close'].shift(1).diff() # Volatility Momentum

# Drop NaN values created by shifting and rolling
df_ticker.dropna(inplace=True)

# Check if we still have enough data after dropping NaNs
if len(df_ticker) < 50: continue

```

Figure 25: Engineering new features based on the past trading week and month prices

As shown in Figure 25, While engineering the new features, the model should only train on past information, so that it doesn't use today's price to guess today's price. This was done by lagging our new 'Log_Close' variable using 'shift' to shift them back by one day and then calculating 5-day (trading week) and 20-day (trading month) moving averages, as well as the daily return (volatility momentum). Any null values caused by the new features were dropped, and another quantity check to ensure that there are at least 50 rows in each ticker was applied.

Creating the Model

- Data splitting and scaling

```

feature_cols = ['Last_Close', 'SMA_5', 'SMA_20', 'Daily_Return']
X = df_ticker[feature_cols]
y = df_ticker['Log_Close']

```

Figure 26: setting the training and testing data

As shown in Figure 26, The first step is to set our feature and target columns, we have 4 features (Last_close, SMA_5, SMA_20, Daily_Return) and our target('Log_Close').

```

split_idx = int(len(X) * 0.8)
X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
y_train, y_test = y.iloc[:split_idx], y.iloc[split_idx:]

```

Figure 27: splitting the training and testing data

As shown in Figure 27, The second step is to split our data for training and testing. We allocated 80% of the dataset for training and 20% for testing, making sure that the data split was in chronological order to avoid any inaccurate predictions.

```
#Data Scaling using standard scaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Figure 28:Scaling input features

As shown in Figure 28, the input features were scaled using standard scaler so the model can evaluate them and train on them fairly, instead of getting distracted by the scale of the numbers. The target variable was left in its log-transformed state to make the model results easier to understand and to simplify the back-scaling process to original price units.

- Model Design

```
model = LinearRegression()
model.fit(X_train_scaled, y_train)
```

Figure 29: Creating the linear regression model

As shown in Figure 29, Our regression model of choice was a Linear Regression Model, after testing other models such as Ridge which adds a penalty to prevent overfitting, we decided to go with Linear Regression as it is a good starting point in financial Forecasting with our filtered dataset, because we can see exactly how the model interprets the relationship between yesterday's and today's price, rather than a "black box" prediction, it also was the best choice in regards to high computational speed and maintaining predictive accuracy.

```
#Predict in log-space
y_log_pred = model.predict(X_test_scaled)
y_pred = np.exp(y_log_pred)
```

Figure 30:Predicting stock prices in log-space

As shown in Figure 30, The model predicts the stock prices in log-space, meaning the prediction is not in dollar prices but the logarithm of that price, which ensures that the model is not looking at absolute price jumps but instead the model is looking at percentage changes. Which also helps enhance our model since linear regression works better with more linear data.

```
#Apply 'exp' to return to original dollar prices
y_actual = np.exp(y_test)
```

Figure 31: back-transforming prediction values

As shown in Figure 31, The log-space prediction is great for the model, however, it is meaningless to humans, since it gives us the log of the price and not the real price. To fix this, we used back-scaling (or back transforming) using the exponential function ‘np.exp’, which converted our mathematical predictions back to dollars, allowing us to use absolute error evaluation metrics

Interpretation of Model Behavior

- Model Evaluation

Table 6: average coefficients and intercept for model

Overall Market Baseline (Avg Intercept): 2.062877		
	Feature	Avg_Coefficient
0	Last_Close	0.433925
1	SMA_5	0.017014
2	SMA_20	0.004847
3	Daily_Return	-0.000030

As shown in Table 6, We extracted the coefficients and intercept of all ticker features for each ticker. We calculated the average intercept of each feature which gave us a result of 2.062877 in log-space which is the model’s starting point, meaning, if all other features were 0 the model would start its prediction at this value for diverse tickers.

By looking at the model weights for each feature, it is clear which feature had the most impact on the prediction results, “Last_Close” (0.433925), this tells us that the model prioritizes the most recently known price. The model then uses the moving averages (0.017014, 0.004847) as clues to see if the stock is generally going up or down. The model almost entirely ignores the daily return (-.000030), which tells us that the model prefers to follow steady trends rather than one day price jumps, this explains why our model is stable and has a low error rate.

Table 7: Model Evaluation report using 4 evaluation metrics

MODEL EVALUATION REPORT	
Total Scanned: 3059 Modeled: 1902	
Avg R2:	0.9407
Avg RMSE:	4116016466.8644
Avg MAE:	2746561933.6693
Avg MAPE:	1.36%

As shown in Table 7, To evaluate the model's performance, we used 4 different evaluation metrics: R-Squared: this metric is considered the "Accuracy Score" for the model's performance, it ranges from 0-1 or 0% - 100% and our model had an average of 0.9407 or 94.07% for all tickers, which is considered very high.

RMSE & MAE: we used these standard metrics in our regression analysis, primarily to highlight how inaccurate they are in our case, since our data set includes a massive variety of stock prices, some at 5\$ while others at 5000\$, so for example, a 10\$ MAE could be a disaster for a low priced stock, and a huge success for highly priced stocks. The results of these metrics were huge numbers which don't apply to our model accurately, Which is why we switched to our next metric MAPE, which analyzes the error percentage, avoiding this scale-dependency.

Mean Absolute Percentage Error (MAPE): for the same reason we formatted our price changes as percentages (high/low stock price differences), we also evaluated our model's percentage error, which tells us how far away our model was from the real price in percentages, in our case the MAPE was is only 1.36%, which means that, on average, the model was 1.36% away from the real price.

- Visualization

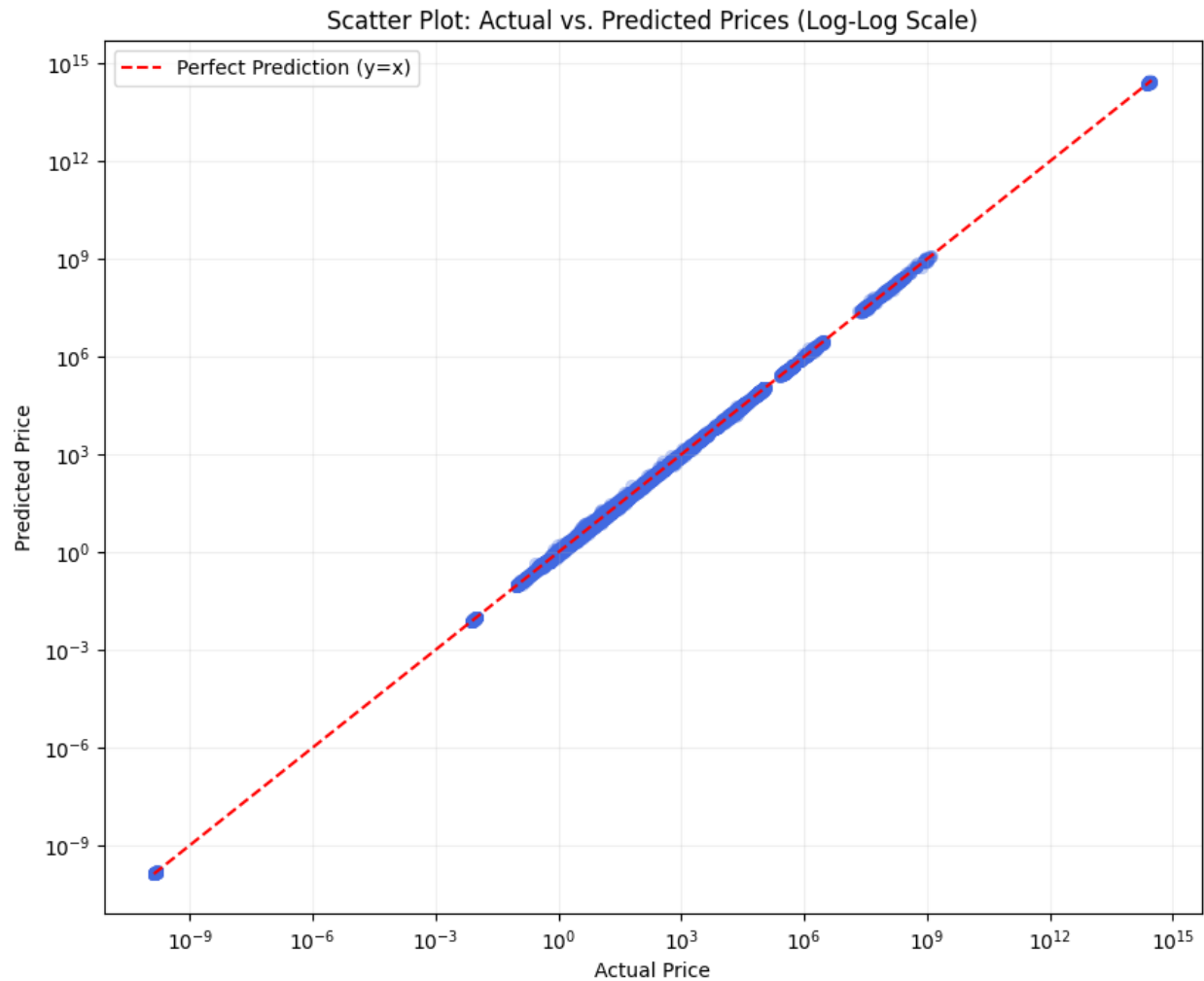


Figure 32: scatter plot with (log-log) scale to show Actual vs. Predicted Prices

As shown in Figure 32, To visually validate the model's performance, we generated a scatter plot, which shows the predicted price vs. the actual price of the stocks, to accommodate the massive price range in our data and clearly display its precision, we used a logarithmic scale.

The result shows a tight cloud of data points, following the "perfect prediction" line. This plot, along with the R2 Score, confirms that the model successfully captured the dynamic nature of the market, without being biased toward any specific price level.

- **Experimental/Alternative Models tested (classification model)**

In addition to the primary Linear Regression Model, we wanted to experiment with the other supervised learning field (classification), although the results were disappointing, they made us more confident in our Linear Regression choice.

To implement classification, we utilized the Hist Gradient Boosting Classifier to predict if the prices were going up or down, or their direction. The model achieved only 45% accuracy. Meaning that while the trend is predictable over time, as shown by our Regression Model Results, the noise in stock data makes the daily stock direction harder to classify. This is why we chose the Regression Model for our project, with the purpose of predicting the continuous values of stock closing prices. For the purposes of this project, we found Linear Regression (utilizing log-transformation), was a more robust approach for financial forecasting with a significantly low error rate (MAPE of 1.36%).

In figure 33 below, you can clearly see that the confusion matrix guesses most of the stock direction predictions being True Positives for each direction, although 45% prediction is better than random chance (33.3%) for our 3 classes, it is insufficient for reliable trading predictions.

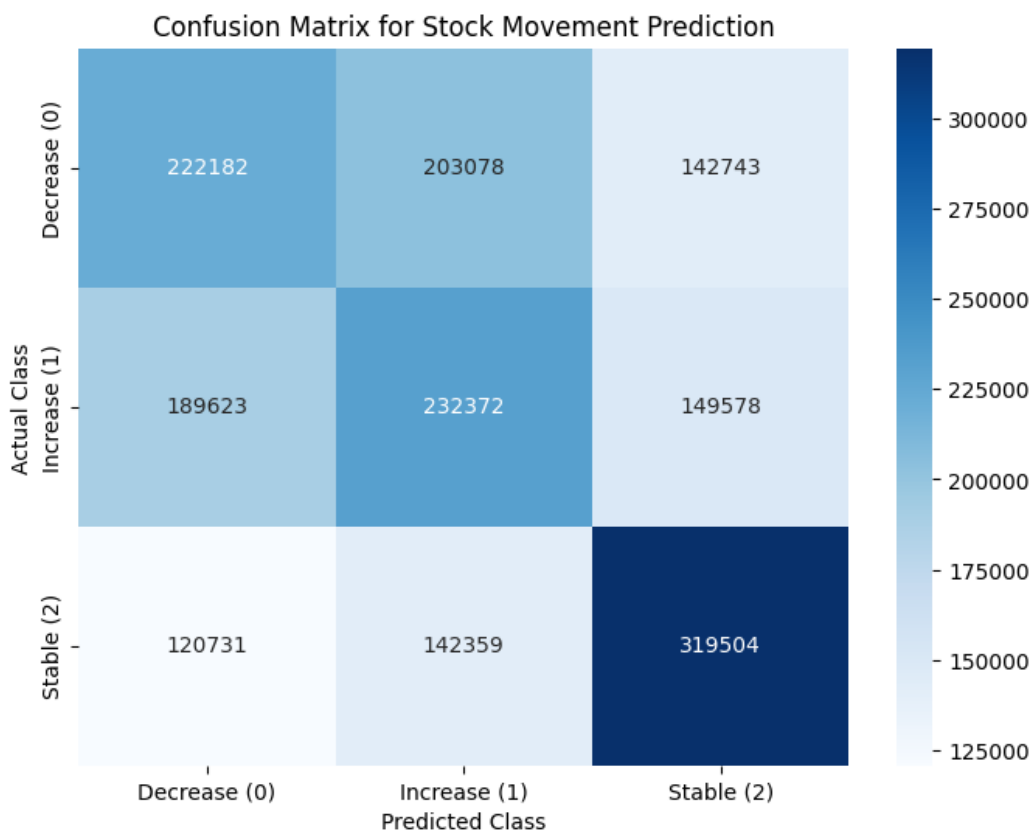


Figure 33: Confusion Matrix for Experimental Supervised Learning (classification) Model

Discussion on the feasibility and limitations of deploying such a system in practice.

- Feasibility

The suggested model based on machine learning is feasible as a decision support tool for analyzing historical stock performance. The system can handle large amounts of historical data as well as large-scale datasets and gain meaningful insights for stock volatility, risk behavior, and the price trends of stocks thanks to the combination of extensive data preprocessing and unsupervised clustering and supervised regression models. Implementing scalable technologies, like K-Means clustering and linear regression, the system can process thousands of stocks efficiently and can be leveraged in offline analytics, commonly used by financial analysts to run deep models that can be exploited by researchers. The system can provide aid to analysts at work; it can discover groups of stocks having the same risk/return characteristics that allow portfolio analysis and market segmentation. It can even be used to predict the price of stocks based on the historical price data, which will lead to exploratory forecasting and comparator-based modeling using a regression-based prediction model. It's based on past numerical data and standard machine learning libraries so it will be flexible and can be adapted to other markets or extended with an additional set of features if larger datasets or more recent ones become available.

- Limitations

Despite its feasibility, the system faces limitations that prevent its use in financial markets. Firstly, the models only work with historical data, which restricts their capacity to fit unexpected market events, economic changes, or shocks that are critical for stock price variations. As a result, predictions should not be regarded as a guarantee but as analytical approximations using past behavior.

Secondly, this dataset had undergone significant preprocessing, including checking for missing data, unusual price entries, outlier values, and discrepancies in the data. The cleaning at this stage leads to a more stable model, but at the same time there are assumptions which may exclude some rare but important market events. Hence, data quality is still an important dependency for system reliability.

Thirdly, the regression model assumes that the relationships between engineered features and future prices remain stable over time; however, in dynamic market environments, this assumption does not always hold true.

Lastly, the system was implemented using offline analysis rather than real-time processing. Although the preprocessing, feature extraction, and model training steps can achieve high performance, deploying the system in a real-time setting would require further optimization and supporting infrastructure. As a result,

real-time forecasting remains challenging due to the computational cost involved, and future work aims to move towards a more integrated and efficient solution.

Nevertheless, while the system is effective for analysis and educational purposes, further development would be required before it could be deployed in real-world financial environments

Conclusion

- Summary

Unsupervised Learning Summary

In the unsupervised learning part of this project, K-Means clustering was applied to group stocks based on their risk characteristics. Two financial features—volatility (standard deviation of returns) and maximum drawdown—were engineered to capture price instability and downside risk. The model was evaluated using the Elbow Method, Silhouette Score, and Davies–Bouldin Index, leading to the selection of three meaningful clusters. The final results provide a clear segmentation of stocks into low-, medium-, and high-risk groups, supporting exploratory analysis of stock behavior without predefined labels.

Supervised Learning Summary

For supervised learning we implemented a regression model, specifically Linear regression, to forecast next day closing prices for all eligible stocks in the dataset after applying quantity and quality filters, which were 1900 tickers. We made our model robust and adaptable to the varying prices of stocks, by making it prioritize price continuity and steady trends over short-term volatility, by using log-transformations, as well as percentage-based error metrics (MAPE).

The model achieved exceptional results with an average prediction error of 1.36%. However, this just shows that the model can capture the market's momentum, but it is only a baseline benchmark. Since we cannot predict the sudden market shifts which often defy historical patterns. The high accuracy of the model is in a controlled environment, which is great as a starting point for research and further analysis, and it is not a guaranteed predictor of future success.

We also Tried a comparative classification model, which predicts stock direction, however, it proved insufficient due to high market noise, which further validates our choice of a regression model.

- Challenges

Throughout the project, several challenges were encountered related to data size, domain knowledge, and team coordination. The stock market dataset contained millions of records, which led to performance issues on some team members laptops, including slow execution and high memory usage during data preprocessing. This required careful dataset reduction to maintain representativeness while ensuring that preprocessing, feature extraction, and model development could be carried out efficiently within available hardware constraints. Additionally, limited prior knowledge of financial concepts such as returns, volatility, and drawdown required additional research and discussion to ensure correct interpretation of the data and appropriate methodological choices. Coordinating tasks among team members with different schedules also presented challenges, which were mitigated through clear task allocation and regular communication.

During the unsupervised learning phase, challenges included the large-scale nature of the data caused significant memory limitations during feature engineering, where iterative processing over individual stocks resulted in long execution times (around 15 minutes) and multiple RAM crashes. Furthermore, extreme values in the dataset led to abnormal clustering behavior, including early visualizations collapsing into a single visible point due to outliers dominating the feature space. After refining the dataset through stock-level aggregation, performance improved; however, additional issues remained, such as clustering plots appearing to show only two clusters despite selecting $k=3$. This occurred because one cluster contained relatively few stocks and was less visually prominent, while overlapping risk characteristics reduced apparent separation in two-dimensional plots. The presence of extreme volatility values also occasionally resulted in very small clusters, highlighting the sensitivity of K-Means to outliers.

On the other hand, different challenges were encountered in the supervised learning, including major problems with long model training times ranging from 30 minutes to over an hour, caused mostly to the quantity of the dataset and the complexity of the models. Additionally, prediction accuracy was affected by large variations in stock prices, as some stocks traded at very low-price levels while others had prices in the thousands. This wide range made it difficult for the model to generalize effectively across all stocks this was solved using log-transformation.

Another challenge involved selecting appropriate evaluation metrics, as prediction performance varied significantly across different price ranges and classes. This required careful consideration of multiple metrics to ensure a fair and meaningful assessment of model performance.

Many of these challenges were mitigated by transitioning to a smaller, more manageable dataset and refining the preprocessing pipeline. This significantly improved computational efficiency, reduced memory issues, and produced more stable and interpretable results across the models.

- Recommendations

To improve future work, it is recommended to utilize more powerful computational resources, such as cloud-based platforms, to handle large-scale stock market datasets more efficiently without extensive data reduction. This would allow models to benefit from a larger historical context and improve overall robustness.

In addition, integrating deeper financial domain knowledge and more advanced financial indicators could enhance feature selection and model interpretability. Gaining further exposure to market concepts or collaborating with finance specialists may lead to more informed modeling decisions.

Finally, improved project coordination practices are recommended for group-based machine learning projects. Clearer documentation, standardized data pipelines, and more frequent communication among team members would support smoother integration of preprocessing, supervised learning, and unsupervised learning components and lead to a more efficient development process.

References

Jallia, F. (2021). *Stock classification using K-means clustering*. **Medium**.

<https://medium.com/@facujallia/stock-classification-using-k-means-clustering-8441f75363de>

MLQ.ai. (2022). *Stock market clustering with K-means*. **MLQ.ai**.

<https://blog.mlq.ai/stock-market-clustering-with-k-means/>

LinkedIn Corporation. (n.d.). *How can you identify patterns in financial data?* **LinkedIn**.

<https://www.linkedin.com/advice/0/how-can-you-identify-patterns-financial-data>

Hudson, R., & Gregoriou, A. (2015). Calculating and comparing security returns is not as simple as you think. *International Review of Financial Analysis*, 41, 1–10. <https://doi.org/10.1016/j.irfa.2014.10.008>

Appendix

Link to the experimental supervised classification model

https://colab.research.google.com/drive/1V2S6yv_tyhcnhxEK5S3tuGjH2bfvVkyn?usp=sharing